EduGenie

Educational AI Assistant

Project Development Phase

This document describes the development phase of the EduGenie project, covering implementation, system modules, AI integration, user interface, data handling, testing preparation, and development milestones.

1. Development Overview

• The development phase converts the approved EduGenie requirements and design into a working educational application.

• Development is performed incrementally, with each major module implemented and tested before complete integration.

• The system is designed to support students and learners through AI-assisted educational features.

2. Development Objectives

• Build a user-friendly educational application.

• Implement the core EduGenie learning features.

• Integrate AI capabilities for educational assistance.

• Provide reliable input validation and error handling.

• Create a modular and maintainable codebase.

• Prepare the application for testing and deployment.

3. Development Environment

• Python development environment.

• Visual Studio Code or another suitable IDE.

• Python virtual environment for dependency isolation.

• Git for version control.

• Environment variables for API credentials and configuration.

• Internet connectivity when external AI services are used.

4. Suggested Project Structure

• EduGenie/

• ■■■ backend/

• ■ ■■■ main.py

• ■ ■■■ routes/

• ■ ■■■ services/

• ■ ■■■ schemas/

• ■ ■■■ utils/

• ■■■ frontend/

• ■ ■■■ app.py

• ■■■ tests/

• ■■■ data/

• ■■■ assets/

• ■■■ .env

• ■■■ .env.example

• ■■■ requirements.txt

• ■■■ README.md

5. Frontend Development

• Develop the EduGenie home screen and navigation.

• Create student-friendly input forms.

• Add learning-topic selection.

• Create areas for questions, explanations, summaries, or other supported learning tasks.

• Display AI-generated educational responses clearly.

• Add loading indicators and user-friendly error messages.

• Maintain a simple and accessible interface.

6. Backend Development

• Create the backend application and API endpoints.

• Define request and response schemas.

• Implement input validation.

• Create service modules for AI and application logic.

• Implement consistent exception handling.

• Configure communication between frontend and backend.

• Keep business logic separate from presentation code.

7. AI Integration

• Create a dedicated AI service module.

• Load API credentials securely from environment variables.

• Build prompts according to the selected educational task.

• Provide learner level, topic, question, and context when available.

• Request clear and educational explanations.

• Handle AI errors, empty responses, and service interruptions.

• Keep AI-provider-specific implementation isolated for maintainability.

8. Educational Features

• Topic-based question answering.

• Concept explanations in learner-friendly language.

• Summarization of educational content.

• Question generation for practice.

• Step-by-step explanations where appropriate.

• Personalized learning assistance based on user-provided level or topic.

• Optional quiz or assessment functionality depending on the final project scope.

9. Data and Validation

• Validate required user inputs before processing.

• Check text length and supported input formats.

• Use structured request models for API communication.

• Avoid unnecessary collection of personal information.

• Use sample or synthetic information during development and demonstrations.

10. Error Handling

• Handle invalid user input without crashing the application.

• Provide understandable validation messages.

• Handle AI API failures and network problems.

• Avoid exposing technical stack traces to users.

• Log appropriate technical information for debugging while avoiding unnecessary sensitive data.

11. Testing During Development

• Create unit tests for validation and utility functions.

• Test backend API endpoints.

• Test AI service behavior with mocked responses where practical.

• Test frontend workflows manually.

• Test error and edge cases.

• Perform integration testing before release.

• Maintain regression tests as features are changed.

12. Security Considerations

• Store API keys outside source code.

• Add .env to .gitignore.

• Validate user-controlled input.

• Protect backend endpoints appropriately for production use.

• Avoid exposing confidential configuration in frontend output.

• Use HTTPS and suitable authentication controls for public deployment where required.

13. Development Milestones

• **Milestone 1:** project structure and environment setup.

• **Milestone 2:** frontend foundation and navigation.

• **Milestone 3:** backend API and validation.

• **Milestone 4:** AI service integration.

• **Milestone 5:** core educational features.

• **Milestone 6:** frontend-backend integration.

• **Milestone 7:** testing, debugging, and documentation.

• **Milestone 8:** deployment-ready build.

14. Development Quality Checklist

• Application starts successfully.

• Dependencies are documented.

• Environment configuration works correctly.

• Frontend communicates with the backend.

• Input validation works.

• AI requests and responses are handled correctly.

• Core educational features work as expected.

• Errors are handled gracefully.

• Automated tests pass.

• No API secrets are committed to source control.

• Documentation is updated.

15. Expected Development Output

• A functional EduGenie MVP.

• A working educational user interface.

• A functional backend API.

• Integrated AI-assisted learning functionality.

• Validated user inputs.

• Error handling and basic security controls.

• Automated and manual testing support.

• Setup and usage documentation.

16. Development Phase Conclusion

• The development phase transforms the EduGenie design into a functional educational AI application.

• A modular architecture makes it easier to test, maintain, and extend the system.

• The completed development build provides the foundation for the testing, documentation, demonstration, and deployment phases.

Note: EduGenie should be used as an educational assistance tool. AI-generated explanations should be reviewed for accuracy, particularly when used for formal academic work.